

Цель работы: получить практические навыки по управлению ОС Linux с помощью systemd.

Теоретическая справка

Самой распространенной системой инициализации ОС Linux является systemd. Она заменила подсистему init и в отличие от последней позволяет реализовать параллельный запуск служб в процессе загрузки системы.

Основной сущностью systemd является – юнит. Юниты бывают нескольких типов:

- `service` — самый используемый вид юнитов, описывает службы или приложения, работающие в фоновом режиме (демоны);
- `socket` — юниты для описания IPC, сетевых сокетов и FIFO буферов;
- `device` — автоматически создаваемые юниты, описывающие файлов устройств, которые необходимы для systemd;
- `mount` — юниты для описания точек монтирования;
- `automount` — юниты для автоматического монтирования файловых систем. Зависят от `mount` юнитов и выполняются после них;
- `swap` — описывает файл подкачки;
- `target` — то, что раньше имело название `runlevel`; юниты, описывающие базовое состояние системы;
- `path` — юниты, для отслеживания изменения путей в файловой системе;
- `timer` — юниты для описания задач по расписанию, аналог `cron`;
- `slice` — юниты для ограничения ресурсов (CPU, mem) для группы процессов с помощью Linux CGroups (`man systemd.slice`);
- `scope` — не конфигурируются с помощью файлов; генерируются автоматически на основе данных из системной шины. Используются для управления группами процессов, созданных извне.

Для управления systemd служат ряд утилит, среди них:

- `systemctl` — для управления загрузкой и получение информации о юнитах, управление загрузкой системы, управление отдельными юнитами;
- `journalctl` — для работы с бинарным журналом systemd;
- `systemctl-analyze` — для получения информации о времени загрузки системы

Если необходимо создать свой сервис, то он должен быть описан через файл юнита `service` в каталоге `/etc/systemd/system`.

Service-файла в systemd обычно состоит из трех секций:

- `[Unit]` — описание сервиса;
- `[Install]` — конфигурационные параметры сервиса;
- `[Service]` — команды сервиса и правила обработки сервиса со стороны systemd.

Справку можно получить в справочных страницах `man`, например так: `man systemd.unit`

Перечислим основные параметры раздела `[Unit]`:

- `Description` — краткое описание юнита;
- `Documentation` — список ссылок на документацию;
- `Before, After` — порядок запуска юнитов;
- `Requires` — если этот сервис активируется, перечисленные здесь юниты будут активированы. Если один из перечисленных юнитов останавливается или падает, этот сервис будет остановлен;
- `Wants` — слабые зависимости. Если один из перечисленных юнитов не может успешно запуститься, это не повлияет на запуск сервиса. Это рекомендуемый способ установления зависимостей;
- `Conflicts` — если установлено что данный сервис конфликтует с другим юнитом, то запуск последнего остановит этот сервис и наоборот.

К основным в секции `[Install]` относятся параметры:

- `Alias` — дополнительные имена сервиса, разделенные пробелами. Большинство команд в `systemctl`, за исключением `systemctl enable`, могут использовать альтернативные имена сервисов;
- `RequiredBy, WantedBy` — данный сервис будет запущен при запуске перечисленных сервисов. Для более подробной информации смотрите описание опций `Wants` и `Requires` в секции `[Unit]`;
- `Also` — определяет список юнитов, которые также будут активированы или деактивированы вместе с данным сервисом при выполнении команд `systemctl enable` или `systemctl disable`.

Основные параметры секции `[Service]`:

- `Type` — настраивает тип запуска процесса;
 - `simple` (по умолчанию) — запускает сервис мгновенно. Предполагается, что основной процесс сервиса задан в `ExecStart`;
 - `forking` — считает сервис запущенным после того, как родительский процесс создает процесс-потомка, а сам завершится;
 - `oneshot` — аналогичен типу `simple`, но предполагается, что процесс должен завершиться до того, как systemd начнет отслеживать состояния юнитов (удобно для скриптов, которые выполняют разовую работу и завершаются);
 - `dbus` — аналогичен типу `simple`, но считает сервис запущенным после того, как основной процесс получает имя на шине D-Bus;
 - `notify` — аналогичен типу `simple`, но считает сервис запущенным после того, как он отправляет systemd специальный сигнал;
 - `idle` — аналогичен типу `simple`, но фактический запуск исполняемого файла сервиса откладывается, пока не будут выполнены все задачи.

- ExecStart — команды вместе с аргументами, которые будут выполнены при старте сервиса. Опция Type=oneshot позволяет указывать несколько команд, которые будут выполняться последовательно. Опции ExecStartPre и ExecStartPost могут задавать дополнительные команды, которые будут выполнены до или после ExecStart;
- ExecStop — команды, которые будут выполнены для остановки сервиса, запущенного с помощью ExecStart;
- ExecReload — команды, которые будут выполнены чтобы сообщить сервису о необходимости пересчитать конфигурационные файлы;
- Restart — когда эта опция активирована, сервис будет перезапущен если процесс прекращен или достигнут timeout, за исключением случая нормальной остановки сервиса с помощью команды systemctl stop;
- RemainAfterExit — если установлена в значение yes (True), сервис будет считаться запущенным, даже если сам процесс завершен. Полезен с Type=oneshot. Значение по умолчанию False;
- User и Group — задает пользователя и группу в контексте безопасности которых будет запущен сервис.

Порядок выполнения работы



Экономьте грант

Выключайте виртуальную машину в случае перерыва в выполнении работы или после завершения.

Часть 1. Получение информации о времени загрузки

1. Выведите информацию о времени, затраченном на загрузку системы
2. Выведите список всех запущенных при старте системы сервисов, в порядке уменьшения времени, затраченного на загрузку сервиса.
3. Выведите список сервисов, запуск которых с необходимостью предшествовал запуску сервиса sshd.
4. Сформируйте изображение в формате svg с графиком загрузки системы, сохраните его в файл на своем компьютере.

Часть 2. Управление юнитами

1. Получите список всех запущенных юнитов сервисов
2. Выведите перечень всех юнитов сервисов, для которых назначена автозагрузка.
3. Определите от каких юнитов зависит сервисы sshd, nginx, postgresql

Часть 3. Создание пробного сервиса

1. Создайте собственный сервис *тутmsg*. Сервис должен:
 - при старте системы записывать в системный журнал дату и время
 - должен запускаться, только если запущен сервис network.
 - запускаться от имени пользователя *тутmsguser*, которого нужно специально создать. При этом сделайте так, чтобы у него не было возможности интерактивного входа.
2. Настройте автоматический запуск сервиса *тутmsg* при старте системы.
3. Запустите сервис.



Примечание

Писать в системный журнал позволяет команда `logger`. Проверить корректность юнит-файла `service` позволяет команда `systemd-analyze`.

После создания нового юнита или редактирования существующего нужно выполнить команду `systemctl daemon-reload` без этого `systemd` продолжает использовать старый кэш конфигурации, и изменения в файле не увидит.

Часть 4. Работа с системным журналом

1. Выведите на консоль системный журнал. Убедитесь, что сервис *тутmsg* отработал корректно.
2. Выведите на консоль все сообщения системного журнала, касающиеся сервиса *тутmsg*.
3. Выведите на экран все сообщения об ошибках в журнале.
4. Определите размер журнала.

Часть 5. Настройка приложения с помощью systemd

Кастомный юнит для nginx

1. Создайте по образцу оригинального юнита `nginx` собственный `service`-юнит *nginx-custom.service* в каталоге `/etc/systemd/system/`.
2. Отключите автозагрузку оригинального юнита.
3. Настройте юнит так, чтобы запуск `nginx` выполнялся с явно указанным конфигурационным файлом (пусть это будет даже стандартный конфиг, мы просто укажем его явно).
4. Обеспечьте автоматический перезапуск сервиса при аварийном завершении.

5. Проверьте корректность юнита, перечитайте конфигурацию `systemd`, затем запустите сервис и включите его автозагрузку.
6. С помощью `systemctl status` убедитесь, что сервис активен и использует требуемую команду запуска.

Ограничение памяти для nginx

1. Создайте unit типа `slice` с именем `nginx.slice`.
2. Ограничьте максимальный объём доступной оперативной памяти для процессов, запускаемых в этом `slice` (пусть это будет 30% доступного ОЗУ).
3. Настройте `nginx-custom.service` так, чтобы он запускался внутри `nginx.slice`.
4. Перезапустите сервис. Убедитесь с помощью `systemctl`, что `nginx` стартует в нужном `slice` и что для этого `slice` установлено ограничение по памяти.
5. Зафиксируйте в отчёте, какое значение памяти было установлено и какими командами вы подтвердили применение ограничения.

Юнит для бэкэнда

Создайте unit, который запускает три процесса node-бэкэнда `librespeed`.

Порядок запуска сервисов

1. Определите точное имя service-юнита PostgreSQL в вашей системе.
2. Настройте `nginx-custom.service` так, чтобы `nginx` запускался только после PostgreSQL и юнита бэкэндов.
3. Сделайте зависимость жёсткой, чтобы при невозможности запуска PostgreSQL запуск `nginx` также не выполнялся.
4. Проверьте результат с помощью просмотра зависимостей и журналов `systemd`.

Таймер для отправки логов на S3

В ЛР №3 был подготовлен скрипт выгрузки файлов на S3. Используйте его для автоматической отправки логов `nginx` в объектное хранилище.

1. Создайте service-юнит `nginx-logs-s3.service`, запускающий существующий скрипт выгрузки логов.
2. Создайте timer-юнит `nginx-logs-s3.timer`.
3. Настройте расписание таймера: понедельник, среда, пятница, 22:00.
4. Сделайте так, чтобы пропущенный запуск выполнялся после включения системы.
5. Включите таймер и проверьте его состояние.

Таймер для резервного копирования PostgreSQL

1. Подготовьте скрипт резервного копирования PostgreSQL, сохраняющий архив в каталог `/var/backups/postgres/`.
2. Создайте service-юнит `postgres-backup.service`, который запускает этот скрипт.
3. Создайте timer-юнит `postgres-backup.timer`.
4. Настройте расписание таймера: вторник, четверг, суббота, 22:00.
5. Включите таймер и убедитесь, что он зарегистрирован в системе и ожидает следующего запуска.



Примечание:

Логические бекапы PostgreSQL могут занимать много места. А мы платим за объёмы хранения. Поэтому нужно сжимать данные перед отправкой на S3.

Общий юнит приложения

1. Создайте unit типа `target` с именем `myapp.target`.
2. Настройте его так, чтобы запуск данного `target` приводил к запуску сервисов `nginx` и PostgreSQL и бэкэндов.
3. Проверьте работу команд `start`, `stop` и `restart` для `myapp.target`.
4. Сравните использование `target`-юнита с использованием обычного `shell`-скрипта для управления группой сервисов.

Вопросы

1. Чем отличаются команды `systemctl restart`, `reload` и `try-restart`?
2. Как с помощью `systemctl` запустить Linux в однопользовательском режиме (НЕ пробуйте это на облачной виртуалке!!)? Для чего это может понадобиться?
3. Пусть вам нужно создать еще один сервис `mysrv`, который не будет запускаться автоматически, и может быть выполнен, только если сервис `mysrv` будет принудительно остановлен уже после старта системы. Приведите параметры и их значения из описания юнитов `mysrv` и `mysrv`, которые обеспечат выполнения этих условий.
4. Как уменьшить размер системного журнала?
5. Как вывести только ошибки и только по определенному сервису?
6. Как переименовать хост средствами `systemd`?
7. Как средствами `systemd` узнать какие пользователи подключены к системе с интерактивным сеансом в настоящий момент? Как отключить конкретного пользователя?

8. Сформулируйте вывод, почему target в systemd лучше подходит для описания состава приложения и его зависимостей. группы сервисов приложения предпочтителен target, потому что он формализует состав приложения, порядок запуска и связи между компонентами лучше, чем внешний shell-скрипт.

Понятийный минимум

1. Назначение и особенности systemd.
2. Типы юнитов systemd.
3. Основные поля юнита сервиса.
4. Основные поля юнита timer.
5. Основные поля юнита target.
6. Основные поля юнита slice.
7. Работа с системным журналом systemd (поиск, фильтрация вывода).